Music Genre Classification Using Frequency Domain Analysis

Anusha A. ; Praharshan R. M.; Shirsha P.; Akash R.; Dhruv B.; Geetika P.

November 21, 2025

1 Introduction

Music genre classification has become an essential component of modern audio processing, enabling applications in music recommendation systems, streaming platforms, automated tagging, and digital library organization. With the rapid growth of online music databases, the need for efficient and accurate classification methods has increased significantly. Traditional approaches that rely on manual labeling or simple time- domain features often fail to capture the complex frequency characteristics that distinguish one genre from another. As a result, frequency-domain techniques have become increasingly important for understanding patterns within musical signals. This study focuses on classifying music genres using frequency-domain analysis, emphasizing the role of the Discrete Fourier Transform (DFT) and related spectral features. By analyzing frequency distributions, energy patterns, and harmonic structures, we aim to identify meaningful characteristics unique to each genre. The goal is to develop a reliable classification framework that leverages spectral features, machine-learning algorithms, and comparative evaluation to enhance genre prediction accuracy across diverse music datasets.

2 Previous Studies and Neural Models

2.1 Early Classical Studies (Feature Extraction and Simple ML Models)

(a) Dannenberg et al. (1997)

1. Introduced one of the earliest machine learning approaches to musical style recognition.
2. Extracted 13 low-level audio features (tempo, loudness, pitch, duration).
3. Classified music using Naive Bayes and Artificial Neural Networks (ANN).
4. Achieved: 98% accuracy for 4 genres
5. Dropped to 77% for 8 genres
6. Important for demonstrating feature-based style classification feasibility.

(b) G. Tzanetakis & P. Cook (2002) — Classical Foundational Paper

1. Proposed three sets of signal-level features for music classification:
2. Timbre features: Short-Time Fourier Transform (STFT)
3. Pitch features: Mel-Frequency Cepstral Coefficients (MFCC)
4. Rhythm features: Wavelet Transform (DWT)
5. Introduced texture windows to group low-level features statistically.
6. Evaluated using: Gaussian Mixture Model (GMM)
7. k-Nearest Neighbor (KNN)
8. Achieved 61% accuracy

2.2 Shift to Deep Learning and Neural Networks

(a) Deng & Yu (2014)

1. General review of Deep Learning applications in Signal Processing.
2. Described how Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) outperform traditional methods in:
3. Speech recognition
4. Audio classification
5. Music information retrieval (MIR)

(b) Rajanna et al. (2015)

1. Applied Deep Neural Networks (DNNs) to music genre classification.
2. Demonstrated high performance by training fully connected DNNs on extracted features (MFCCs, chroma).
3. Highlighted robustness of deep models over classical SVM/GMM.

(c) Sigtia, Dixon & S. (2014)

1. Developed Convolutional Neural Network (CNN) for feature learning from music spectra.
2. Input: STFT spectrograms
3. Architecture: 500 neurons, 3 hidden layers.
4. Combined CNN-extracted features with classical classifiers.
5. Result: 83% accuracy on GTZAN dataset.

(d) Tang et al. (2018)

1. Used Hierarchical Long Short-Term Memory (LSTM) model.
2. Multi-level structure: divided genres into strong and mild, then into subgenres.
3. Each LSTM block handled sub-classification recursively.
4. Accuracy: 50%.
5. Important for modeling temporal dependencies (music sequence context).
6.

(e) Zhang et al. (2016)

1. Proposed improved CNN for genre classification:
2. Combined max pooling and average pooling
3. Added shortcut connections (skip layers)
4. Increased efficiency of feature extraction and overall accuracy.

2.3 Hybrid Ensemble Approaches

1. Introduced ensemble of multiple models: KNN, SVM, etc
2. Used soft voting among models → improved overall classification F1-score to 0.78 (78
3. This hybrid approach outperformed:
4. GenreXpose logistic regression (71%)
5. Pure CNN-based system (46%).
6.

3 Signal Features used in Analysis of Music Signals:

There are many ways to analyze signals, using signal features. These are used in Signal Processing, Speech Recognition, and Audio analysis. Our focus is on Music Signals, so we will be applying these features on music. When music is analyzed on the basis of signals, the waveforms are converted to more quantitative features to enable genre detection. Dominating frequencies (spectral properties), sound evolution (temporal properties), and specifics that are noticed by the human ear (perceptual cues), are some quantitative descriptions for detecting the genre of music through signal analysis.

3.1 ZCR: Zero Crossing Rate

It is essentially a measure of how many times a signal crosses the zero axis over a given time interval. Indicates the number of times the amplitude of a signal changes from positive to negative, or vise versa. A higher ZCR corresponds to a higher frequency content, implying rapidly changing signals.

$$\text{ZCR} = \frac{1}{T-1} \sum_{n=1}^{T-1} 1_{R_{<0}}(x[n] \cdot x[n-1]) \tag{1}$$

Where, $x[n]$ is the signal amplitude at time n.

Therefore, in context to music analysis, rapid oscillations can refer to high-frequency components, such as electric guitar, EDM, rock, metal, and other such music. Generally, music which consists of distortions or noise, per se, would have a <u>higher ZCR</u>.

Smoother waveforms that do not have such rapid/frequent oscillations can refer to vocals, classical, jazz, and other such smooth-flowing music. These have a <u>lower ZCR</u>.

Pop music, due to how dynamic and variable it is, with much freedom of instruments and music patterns, would have a <u>medium ZCR</u>.

Also, ZCR can have use in separating vocal, smooth music from noisy, beat-heavy genres, when averaged over a short time interval, such as 20-30ms.

3.2 MFCC: Mel-Frequency Cepstral Coefficients

MFCC helps capture the timbre, or texture of sound as humans hear it. The way the human ear can tell the difference between the same note played on a piano versus the guitar, is what MFCC can describe quantitatively.

As humans do not perceive frequencies linearly:

- Below 1kHz, small changes are perceived clearly
- Above 1kHz, the difference has to be larger to be noticeable

Hence, the Mel scale expands lower frequencies and compresses higher frequencies to imitate this perception. Following are the Steps involved in MFCC:

3.2.1 Pre-Emphasis

Before analysis of the signal, higher frequencies are slightly amplified, due to the natural attenuation of high notes by microphones and vocal cords.

$$y[n] = x[n] - \alpha \cdot x[n-1], \quad \alpha \approx 0.97 \tag{2}$$

3.2.2 Framing

The signals are sliced into short frames, small enough to be perceived as stationary for further analysis.

3.2.3 Windowing

A hamming window is applied to each frame. Smoothes edges to avoid spectral leakage in the next step. This is required because the next step is FFT (Fast Fourier Transform), which when applied on a short frame of signals, assumes that the signal repeats forever in time. However, if the frame ends abruptly, the beginning and the end of the signal fragment do not line up, which causes <u>artificial high-frequency</u> components in the spectrum. This is called <u>spectral leakage</u>.

Hamming Window:

$$w[n] = 0.54 - 0.46\cos\left(\frac{2\pi n}{N-1}\right), \quad 0 \le n \le N-1 \tag{3}$$

N = Length of Frame
$w[n]$ = window coefficient, lies between 0 and 1

3.2.4 FFT (Fast Fourier Transform)

Converts time domain frames into frequency domain. This provides the magnitude spectrum, that represents which frequencies are present, how strong, etc.

$$X[k] = \sum_{n=0}^{N-1} x[n] \cdot e^{-j\frac{2\pi kn}{N}}, \quad k = 0, 1, \ldots, N-1 \tag{4}$$

3.2.5 Mel Filter Bank

Filters are still required to convert the FFT signals into something that can be perceived by the human ear. The Mel Scale maps physical frequencies (Hz) to perceived frequencies (Mels), by the following formula:

$$M(f) = 2595 \cdot \log_{10}\left(1 + \frac{f}{700}\right) \tag{5}$$

It compresses the higher frequencies and expands the lower frequencies. The Mel Filters are triangular filters that cover a band of frequencies and compute the energy in the specific band over the Mel Scale. The triangular nature of these bands allows for a smooth representation and prevents sudden jumps.

$$H_m[k] = \begin{cases} 0 & k < f[m-1] \\ \frac{k - f[m-1]}{f[m] - f[m-1]} & f[m-1] \le k \le f[m] \\ \frac{f[m+1] - k}{f[m+1] - f[m]} & f[m] < k \le f[m+1] \\ 0 & k > f[m+1] \end{cases} \tag{6}$$

The energy in each Mel band:

$$E[m] = \sum_{k=0}^{N-1} |X[k]|^2 \cdot H_m[k] \tag{7}$$

The purpose of the Mel filter is to condense many FFT fragments into perceptual bands, as FFT gives the output in much detail, which is unnecessary and many are barely perceivable by the human ear. It smooths the spectrum, allows for MFCC to reflect the texture of the sound, and makes the MFCC feature robust towards minute noise changes.

Generally, energy concentrated in the lower Mel bands represent Classical music, while Rock/Metal have high-energy band concentration. Pop lies in between and has a smoother distribution, while EDM has periodic bursts into the high-energy Mel bands.

3.2.6 Logarithm of Energy

Apply log on each filter output.

$$S[m] = \log(E[m]) \tag{8}$$

Loudness is perceived logarithmically by the human ear.

3.2.7 Discrete Cosine Transform (DCT)

Decorrelation and compression of the log energies using DCT, which gives the MCC coefficients.

$$c[n] = \sum_{m=1}^{M} S[m] \cos\left[\frac{\pi n}{M}\left(m - \frac{1}{2}\right)\right], \quad n = 1, 2, \ldots, K \tag{9}$$

Generally, 13 coefficients are considered, where $c(0)$ represents overall energy/loudness, c(1-5) represents a broader spectral scope, and c(6-13) represents the finer spectral details. Therefore, the shape is described.

3.2.8 Patterns observed:

Classical music tends to have smoother, slow varying MFCC coefficients, while rock and metal have higher fluctuations. EDM has periodic spikes corresponding to beat drops, while pop music has repetitive structures.

3.3 Spectral Centroid:

This feature helps locate, essentially, the "centre of mass" of the spectrum, and speaks about the brightness of the sound. Hence, it directly depends on the mixing style and the instruments of the genre.

$$\text{Spectral Centroid} = \frac{\sum_{k=0}^{N-1} f[k] \cdot |X[k]|}{\sum_{k=0}^{N-1} |X[k]|} \tag{10}$$

Where $f[k]$ is the frequency at bin k and $|X[k]|$ is the magnitude of the FFT at bin k.

Higher frequency energy has a high centroid, while low frequency energy has a low centroid. It is useful in music analysis due to the fact that a high centroid generally represents Rock/Metal music, a low centroid for Classical, and EDM tends to have a fluctuating centroid.

4 Classification Algorithms

Once the feature set has been extracted, the next step is to assign each new audio sample to a genre using a suitable classification method. Several algorithms can be employed for this task; however, this section focuses on two of the most widely used approaches.

4.1 K-Nearest Neighbours (KNN)

K-Nearest Neighbours (KNN) is a supervised machine learning algorithm primarily used for classification but can also be applied to regression tasks. It works by finding the k closest data points (neighbours) to a given input and making a prediction based on:

- the majority class (for classification), or
- the average value (for regression).

KNN uses distance metrics to identify the nearest neighbours. The commonly used distance metrics are:

1. **Euclidean Distance**:
$$d_{\text{euclid}}(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$
2. **Manhattan Distance**:
$$d_{\text{manhattan}}(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$
3. **Minkowski Distance**:
$$d_p(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

A sample Python implementation and output are shown in the figures.

4.2 Support Vector Machine (SVM)

The Support Vector Machine (SVM) is a supervised learning algorithm that is used for both classification and regression tasks. It works by finding the optimal boundary, known as a **hyperplane**, that separates different classes in the data.

The goal of SVM is to:

"Find the hyperplane that best separates two classes by maximizing the margin."

When data is not linearly separable, SVM uses **kernel functions** to map it into a higher-dimensional space where separation becomes possible. A kernel computes inner products in this higher-dimensional space without explicitly mapping the coordinates.

The equation of a linear hyperplane is:

$$\omega^\top x + b = 0$$

where:

- ω is the normal vector,
- b is the offset from the origin.

Prediction for a new point x is given by:

$$\hat{y} = \begin{cases} 1, & \text{if } \omega^\top x + b \geq 0 \\ -1, & \text{if } \omega^\top x + b < 0 \end{cases}$$

4.3 Artificial Neural Networks (ANN)

Artificial Neural Networks (ANNs) are computational models inspired by biological neural networks. They consist of interconnected artificial neurons that process information, learn patterns, and make predictions.

4.4 Gradient Boosting

Gradient Boosting is an ensemble learning technique where each new model is trained to minimize the loss function of the previous model using gradient descent.

At each iteration:

1. The gradient of the loss function with respect to predictions is computed.
2. A weak learner is trained to reduce this gradient.
3. The new model's predictions are added to the ensemble.

This process continues until a stopping criterion is satisfied.
graphicx booktabs caption subcaption siunitx

5 Implementation and Results

5.1 Dataset

The GTZAN Genre Collection was used as the dataset. It contains:

- 1000 audio clips,
- 10 genres (100 clips each),
- 30-second `.wav` files.

The genres included: *Blues, Classical, Country, Disco, HipHop, Jazz, Metal, Pop, Reggae,* and *Rock.*

5.2 Preprocessing

Before training, all audio files were preprocessed to ensure uniformity:

- Conversion to mono,
- Resampling to 22,050 Hz,
- Normalization of amplitude,
- Automatic skipping of corrupted files (a known issue in GTZAN).

These steps ensured that all audio samples were consistent for feature extraction.

5.3 Feature Extraction

We extracted **26 MFCC summary features** for each audio file:

- 13 MFCC means,
- 13 MFCC standard deviations.

MFCCs capture characteristics such as timbre and frequency shape and are effective for genre classification.

Final dataset

The final dataset (`gtzan_features.csv`) contained:

- Approximately 990 valid samples (after removing corrupted files),
- 27 columns (26 MFCC features + genre label).

5.4 Evaluation Metrics

Model performance was measured using:

- Accuracy,
- Confusion matrix,
- Precision, Recall, F1-score (via classification report).

5.5 Results

5.5.1 Accuracies

The following table summarises the test-set accuracies for each model:

Table 1: Model Accuracies (test set)

Model	Accuracy (%)
KNN	64.00
SVM	67.50
ANN	68.50
Gradient Boosting	61.00

The ANN achieved the best performance at 68.5.

5.5.2 Best Model Classification Report (ANN)

Key observations from the ANN classification report:

- **Classical**: F1 ≈ 0.90 — very distinct MFCC patterns.
- **Metal**: F1 ≈ 0.85 — strong high-frequency content captured in MFCCs.
- **Pop**: F1 ≈ 0.79, **Jazz**: F1 ≈ 0.71 — perform well.
- **Rock**: F1 ≈ 0.43 — poor, often confused with Metal and Country.
- **Disco**: F1 ≈ 0.51 — overlaps with Pop.

Macro average F1 is approximately 0.68, supporting ANN as the best overall model.

5.6 Discussion

Why ANN performed best

- ANNs model complex nonlinear relationships present in MFCC features.
- Hidden layers let the classifier learn subtle genre differences.
- Neural networks handle moderate dataset sizes ($\sim$1000 clips) well.

Why Gradient Boosting performed worst

- MFCCs alone may not capture sufficient rhythmic and spectral detail.
- Ensemble trees can struggle with high-dimensional continuous MFCC features.

Genre Confusions

- *Rock* vs *Metal* vs *Country*: heavy overlap in MFCC feature space.
- *Pop* vs *Disco*: similar vocal and percussion features.
- *Reggae* sometimes confuses with *Country* due to rhythmic similarities.

5.7 Limitations

- Using only MFCCs limits overall classification performance.
- Adding features such as Zero-Crossing Rate (ZCR), Spectral Centroid, Chroma, and Tempo could improve accuracy.
- GTZAN contains known mislabeled and corrupted samples, which affect performance.

5.8 Conclusion

The ANN emerged as the most effective classifier with an accuracy of 68.5, outperforming KNN, SVM, and Gradient Boosting. MFCC-based features provide a solid baseline for genre classification, but incorporating additional spectral and rhythmic descriptors would likely yield better results.

```
import librosa, numpy as np, matplotlib.pyplot as plt

y, sr = librosa.load('song.wav')
zcr = librosa.feature.zero_crossing_rate(y, frame_length=2048,
hop_length=512)[0]
times = librosa.times_like(zcr, sr=sr, hop_length=512)
plt.figure(figsize=(12,4))
plt.plot(times, zcr, label='ZCR')
plt.title('Zero Crossing Rate over Time')
plt.xlabel('Time (s)')
plt.ylabel('ZCR')
plt.legend()
plt.show()
```

Figure 1: ZCR: Produces a waveform curve where high spikes indicate percussion/noise bursts and flat regions indicate sustained tunes

```
import librosa
import numpy as np
import matplotlib.pyplot as plt

y, sr = librosa.load('song.wav', sr=22050)

mfccs = librosa.feature.mfcc(y=y, sr=sr, n_mfcc=13)
librosa.display.specshow(mfccs, x_axis='time', sr=sr)
plt.title('MFCC (13 coefficients)')
plt.ylabel('Coefficient index')
plt.xlabel('Time (s)')
plt.colorbar(label='Amplitude')
plt.show()
```

Figure 2: The output is a plot that has time on the x-axis, MFCC coefficients on the y-axis, and the colour shows intensity, hence representing the timbre and rhythm of the song.

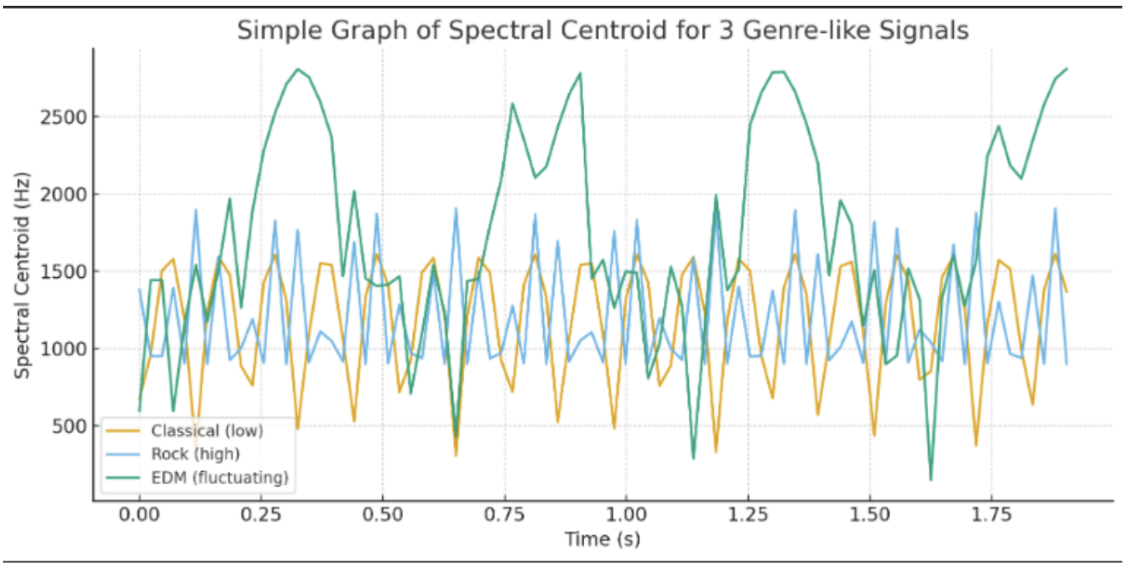

Figure 3: Spectral Centroid: Graph

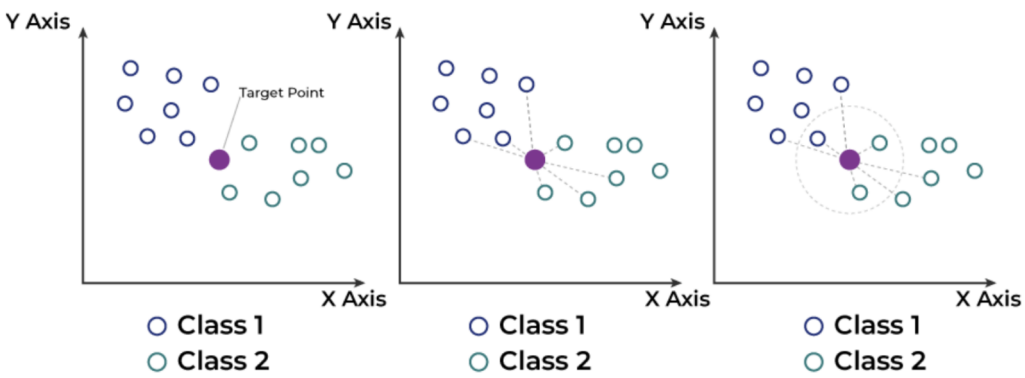

Figure 4: Demonstration of the K-Nearest Neighbours algorithm.

```python
import numpy as np
from collections import Counter
#using euclidean distance
def euclidean_distance(point1, point2):
    return np.sqrt(np.sum((np.array(point1) - np.array(point2))**2
        ))
#traning the model
def knn_predict(training_data, training_labels, test_point, k):
    distances = []
    for i in range(len(training_data)):
        dist = euclidean_distance(test_point, training_data[i])
        distances.append((dist, training_labels[i]))
    distances.sort(key=lambda x: x[0])
    k_nearest_labels = [label for _, label in distances[:k]]
    return Counter(k_nearest_labels).most_common(1)[0][0]
 #example training data
training_data = [[1, 2], [2, 3], [3, 4], [6, 7], [7, 8]]
training_labels = ['A', 'A', 'A', 'B', 'B']
test_point = [4, 5]
k = 3
prediction = knn_predict(training_data, training_labels,test_point
    , k)
print(prediction)
```

Figure 5: Python implementation of the K-Nearest Neighbours algorithm.

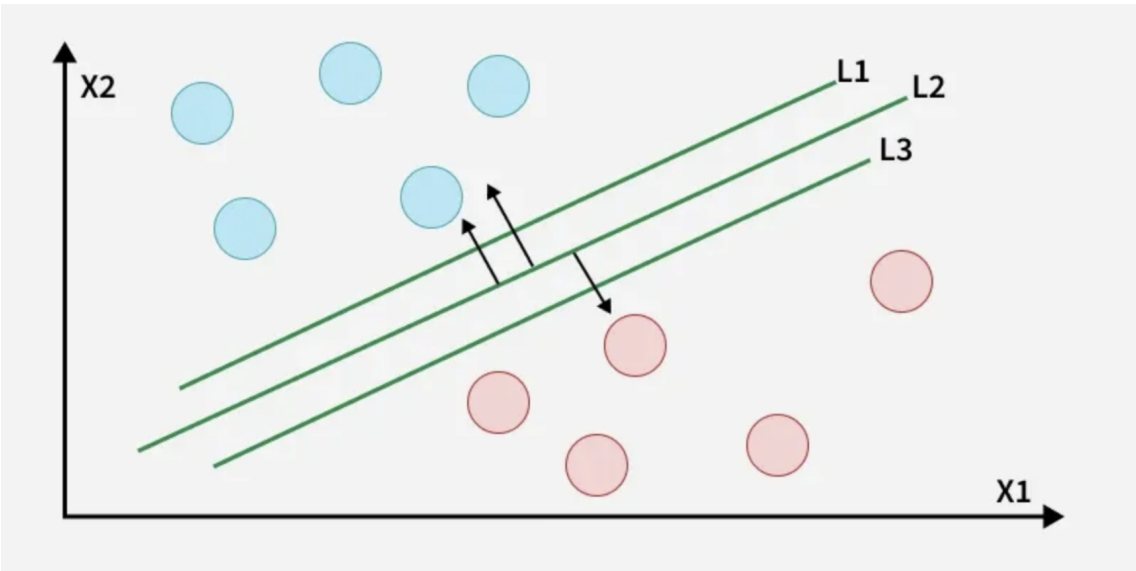

Figure 6: Demonstration of Support Vector Machine

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# ------------------------------------
# LOAD CSV
# ------------------------------------
csv_path = r"C:\Users\Shirsha Pattanaik\gtzan_features.csv"
df = pd.read_csv(csv_path)

print("Dataset Loaded Successfully!")
print(df.head())

# ------------------------------------
# SEPARATE FEATURES & LABELS
# ------------------------------------
X = df.drop("genre", axis=1)
y = df["genre"]

# ------------------------------------
# TRAIN / TEST SPLIT (80/20)
# ------------------------------------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# ------------------------------------
# STANDARDIZE FEATURES
# ------------------------------------
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ------------------------------------
# 1. KNN
# ------------------------------------
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
knn_pred = knn.predict(X_test)
knn_acc = accuracy_score(y_test, knn_pred)

# ------------------------------------
# 2. SVM (RBF kernel)
# ------------------------------------
svm = SVC(kernel="rbf", gamma="scale", C=10)
svm.fit(X_train, y_train)
svm_pred = svm.predict(X_test)
svm_acc = accuracy_score(y_test, svm_pred)
```

Figure 7: (a) Model Training and Implementation

```python
# ------------------------------------
# 3. ANN (MLPClassifier)
# ------------------------------------
ann = MLPClassifier(hidden_layer_sizes=(128, 64), max_iter=500)
ann.fit(X_train, y_train)
ann_pred = ann.predict(X_test)
ann_acc = accuracy_score(y_test, ann_pred)

# ------------------------------------
# 4. Gradient Boosting
# ------------------------------------
gbt = GradientBoostingClassifier()
gbt.fit(X_train, y_train)
gbt_pred = gbt.predict(X_test)
gbt_acc = accuracy_score(y_test, gbt_pred)

# ------------------------------------
# PRINT ACCURACIES
# ------------------------------------
print("\n=== MODEL ACCURACIES ===")
print(f"KNN Accuracy: {knn_acc * 100:.2f}%")
print(f"SVM Accuracy: {svm_acc * 100:.2f}%")
print(f"ANN Accuracy: {ann_acc * 100:.2f}%")
print(f"Gradient Boosting Accuracy: {gbt_acc * 100:.2f}%")

# ------------------------------------
# PRINT CONFUSION MATRICES
# ------------------------------------
print("\n=== CONFUSION MATRIX: SVM ===")
print(confusion_matrix(y_test, svm_pred))

print("\n=== CONFUSION MATRIX: GBT ===")
print(confusion_matrix(y_test, gbt_pred))

# ------------------------------------
# PRINT CLASSIFICATION REPORT
# ------------------------------------
print("\n=== CLASSIFICATION REPORT: BEST MODEL ===")
if svm_acc >= max(knn_acc, ann_acc, gbt_acc):
    print("Best Model: SVM\n")
    print(classification_report(y_test, svm_pred))
elif gbt_acc >= max(knn_acc, svm_acc, ann_acc):
    print("Best Model: Gradient Boosting\n")
    print(classification_report(y_test, gbt_pred))
elif ann_acc >= max(knn_acc, svm_acc, gbt_acc):
    print("Best Model: ANN\n")
    print(classification_report(y_test, ann_pred))
else:
    print("Best Model: KNN\n")
    print(classification_report(y_test, knn_pred))
```

Figure 8: (b) Model training and Implementation

```python
import pandas as pd
from scipy.io import savemat

# Load CSV
df = pd.read_csv(r"C:\Users\Shirsha Pattanaik\gtzan_features.csv")

# Separate features & labels
features = df.drop("genre", axis=1).values
labels = df["genre"].values

# Save to MAT file
savemat("gtzan_data.mat", {
    "features": features,
    "labels": labels
})

print("MAT file saved as gtzan_data.mat")
```

Figure 9: Python Implementation: Required data formatting and label conversion

```python
# ====================================================
# GTZAN FEATURE EXTRACTION SCRIPT (ERROR-SAFE VERSION)
# - Skips corrupted WAV files
# - Uses soundfile + scipy + python_speech_features
# - Saves MFCC features to gtzan_features.csv
# ====================================================

import os
import glob
import numpy as np
import soundfile as sf
from scipy.signal import resample_poly
from python_speech_features import mfcc
import pandas as pd

# ----------------------------
# 1. AUDIO PREPROCESSING
# ----------------------------
def preprocess_audio(path, target_sr=22050):
    """
    Loads audio, converts to mono, resamples, normalizes.
    """
    audio, sr = sf.read(path, dtype='float32')

    # Convert stereo -> mono
    if audio.ndim > 1:
        audio = np.mean(audio, axis=1)

    # Resample if needed
    if sr != target_sr:
        g = np.gcd(sr, target_sr)
        up = target_sr // g
        down = sr // g
        audio = resample_poly(audio, up, down)
        sr = target_sr

    # Normalize amplitude
    audio = audio / (np.max(np.abs(audio)) + 1e-12)

    return audio, sr
```

Figure 10: (a)Python Code Implementation of audio preprocessing and feature extraction

```python
# ----------------------------
# 2. EXTRACT MFCC FEATURES
# ----------------------------
def extract_features(audio, sr, n_mfcc=13):
    """
    Computes MFCCs and returns mean + std (26 features).
    """
    mfcc_feat = mfcc(
        signal=audio,
        samplerate=sr,
        numcep=n_mfcc,
        winlen=0.032,
        winstep=0.016,
        nfilt=26,
        nfft=1024   # larger FFT to avoid warnings
    )

    mfcc_mean = np.mean(mfcc_feat, axis=0)
    mfcc_std = np.std(mfcc_feat, axis=0)

    return np.hstack([mfcc_mean, mfcc_std])  # 26 features

# ----------------------------
# 3. PROCESS ALL FILES IN GTZAN
# ----------------------------
def process_gtzan(root_folder):
    genres = sorted(os.listdir(root_folder))
    all_data = []

    for genre in genres:
        genre_path = os.path.join(root_folder, genre)
        wav_files = glob.glob(os.path.join(genre_path, "*.wav"))

        print(f"Processing {genre} ({len(wav_files)} files)...")

        for fpath in wav_files:
            try:
                audio, sr = preprocess_audio(fpath)
            except Exception as e:
                print("Skipping corrupted file:", fpath)
                continue

            feats = extract_features(audio, sr)

            # Store features + genre label
            row = list(feats) + [genre]
            all_data.append(row)
```

Figure 11: (b)Python Code Implementation of audio preprocessing and feature extraction

```python
    # Column names
    columns = (
        [f"mfcc_mean_{i}" for i in range(13)] +
        [f"mfcc_std_{i}" for i in range(13)] +
        ["genre"]
    )

    df = pd.DataFrame(all_data, columns=columns)
    return df

# ----------------------------
# MAIN EXECUTION
# ----------------------------
if __name__ == "__main__":
    root = r"C:\Users\Shirsha Pattanaik\Downloads\archive\Data\genres_original"
    df = process_gtzan(root)

    df.to_csv("gtzan_features.csv", index=False)
    print("\n✔ Feature extraction complete!")
    print("✔ Saved CSV: gtzan_features.csv")
```

Figure 12: (c)Python Code Implementation of audio preprocessing and feature extraction